Republic of the Philippines
Polytechnic University of the Phlippines - Sta. Mesa
Department of Computer Engineering
College of Engineering and Architecture

DANAS

Digital Academy for Navigating Academic, Skills and Success

A Learning Management System

Project Title:

DANAS — Learning Management System

Group Members:

Bernard Anthony F. Orquez — Project Manager
Jayvee Rayniel B. Iggo — Backend Developer
Ella Gajardo — Backend Developer
Charles Irwin Garado — Backend Developer
Krysha Loraine F. Fajura — Backend Developer
Shan Ashley C. Carillo — UI/UX Designer
Marcus Alexander S. Rosales — UI/UX Designer
Verdhenz Apple M. Domasig — UI/UX Designer
Erwin Jhay M. Santos — Quality Assurance/Documentation
Osiris Racaza Jugo — Quality Assurance/Documentation

Course:

Computer Engineering

Instructor:

Engr. Godofredo T. Avena

Submission Date:

June 15, 2026

Abstract

DANAS (Digital Academy for Navigating Academic Skills and Success) is a desktop-based Learning Management System built in Python with object-oriented programming as a framework and Tkinter for the graphical user interface. It is a system designed for instructors to upload and manage their content, while students can enroll in classes and browse through courses.

The leading objective of this project is to apply concepts in OOP, such as inheritance, polymorphism, and encapsulation. This framework allows for high integrity of file handling for data longevity and creates ease of use and access without technical issues comparable to most modern LMS platforms.

Key findings suggest that a class-based system structure highly benefits the scalability and maintainability of code. All essential features of a standard LMS are implemented within the system, such as course management, enrollment management, and PDF upload and download functionality.

The project concludes that Python and Tkinter are sufficient tools for building a fully functional LMS prototype, while the proper implementation of OOP design patterns reduces the difficulty of developing the project for a team of this size.

1. Introduction

1.1 Background

Learning Management Systems are a core part of education, especially for those in higher education. More than just a tool for learning, it makes education more accessible, providing a platform for delivering source materials digitally, straightforward enrollment, and a centralized hub for academic content. LMS like Canvas, Moodle, and Blackboard are notable for demonstrating well-structured, optimized systems. This project was developed as an academic exercise to replicate a fully functional LMS using Python.

1.2 Problem Statement

Managing a classroom can be difficult, especially when everything is done manually using papers and folders. It can be hard to keep track of student records, course materials, and enrollment information, and sometimes files can get lost or disorganized. Because of these problems, our group decided to create a desktop-based Learning Management System (LMS) using Python.

The main goal of our project was to make a simple and user-friendly application where teachers and students can manage courses, registrations, and files in one place.

1.3 Project Objectives

This project aims to:

- **Apply Object-Oriented Programming (OOP):** Implement fundamental OOP concepts such as classes, inheritance, and polymorphism to structure the system logically.
- **Implement File Handling & Data Persistence:** Learn and apply file handling in Python to save, retrieve, and manage course records and PDF documents securely.
- **Develop a Graphical User Interface (GUI):** Design and build an intuitive, user-friendly desktop interface using Tkinter.
- **Practice Core Software Engineering Principles:** Understand and implement modularity, encapsulation, and clean code structure in a practical, student-level group application.

1.4 Scope

Our system focuses on the basic features of a Learning Management System (LMS). It allows users to add, edit, and delete courses, as well as enroll or remove students from classes. The system also supports uploading and downloading PDF files for course materials.

To save data, we used local file storage with a text file so the information can still be accessed even after closing the application. Since this is a beginner-level project, the system only works as a desktop application and does not use online databases or cloud storage yet.

2. Methodology

2.1 Research Design

- This project used a development-based approach in creating a Student Portal for managing course modules. The system was planned, designed, developed, and tested to ensure that it met the needs of students. The design was inspired by the PUP Learning Management System (LMS) to provide a familiar and user-friendly experience. The project focused on creating a simple platform where students can easily access, view, and download their learning materials.

2.2 Tools Used

Tool	Purpose
Python	Main programming language
Tkinter	Graphical User Interface (GUI)
SQLite	Database Management
Visual Studio Code (VSCode)	Code Editor
Canva	More visualizing in the GUI
Diagram	Flowchart
GitHub	Updating Codes

3. System Design and Implementation

3.1 System Overview

1. Course Management:

- **Different Course Materials / Structuring:** Implement a structure where instructors can initialize new subjects by specifying a unique Course ID, Title, and attaching a primary PDF lecture material.
- **Course Customization Engine:** Use object-oriented programming to handle course data tracking, allowing instructors to seamlessly edit text descriptions or swap/overwrite existing lecture PDFs when course details change.
- **Course Removal Operations:** Allow instructors to completely remove an entire course structure and its database records from the active system repository.

2. Enrollment Features:

- **Browse Available Courses:** Provide an open, accessible catalog display listing all actively running course blocks created in the system.
- **Enroll in Course:** Allow students to select a course from the available directory and register instantly, binding that course ID to their student profile track.
- **Unenroll/Withdraw from Course:** Give students full autonomy to drop out or securely withdraw their access permissions from any actively enrolled subject frame.
- **View Enrolled Courses (Dashboard):** Display a tailored summary panel tracking current loads—showing a personalized schedule for students and a class roster overview for instructors.

3. PDF Management:

- **Upload PDF Engine:** Automatically process external files imported by instructors and mirror them cleanly inside a designated system project folder (`course_materials`).

- **Download PDF Utility:** Grant enrolled students immediate access to safely export and save copy-tracked lecture files out of the program directly onto their local machine storage.

4. File Operations:

- **Save to File:** Automatically save all system parameters (courses, student directories, and enrollment mappings) to local text or CSV files whenever a course is added, edited, or an enrollment change occurs.
- **Load from File:** Read and parse stored text or CSV data records automatically at application startup, ensuring that user progress and course structures persist safely between sessions.

5. User Interface:

- Develop a multi-panel desktop GUI using Tkinter that includes clean input forms for entering course details, intuitive buttons for submitting actions, interactive lists for navigating courses, and clear options for uploading and downloading lecture files.

3.2 Key components

The system was created using Python and the Tkinter library for the user interface. It follows Object-Oriented Programming (OOP) to keep the code organized and easier to manage.

The system is divided into several main components:

- **User Interface (UI)** – Handles the layout of the portal, including tabs, buttons, and module display.
- **Course Management** – Manages the creation, editing, and organization of courses.
- **Module/File Handling** – Responsible for uploading, storing, and downloading PDF learning materials.
- **Database System (SQLite)** – Stores course and module information so data can still be accessed even after closing the program.

Each part of the system is separated so it is easier to update, fix, or improve without affecting the whole program.

3.3 Implementation Process

The system works by allowing instructors to create courses and upload PDF modules into the system. These files are stored locally and linked to the course information saved in the SQLite database.

When a student opens the portal, the system loads the available courses and displays them in the interface. From there, students can select a course and view or download the attached module files.

The program uses an OOP structure, which means each function, such as course handling, file management, and interface design, is separated into distinct parts of the code. This helps keep the system organized and easier to maintain.

The user interface was designed to be simple and clean. It uses scrollable sections and clear buttons, allowing users to easily navigate the portal and access their learning materials without confusion.

4. Testing and Results

4.1 Testing methods

The development team evaluated the DANAS Learning Management System through manual functional testing and graphical user interface (GUI) testing. Upon testing, essential features like course management, student enrollments, and downloading/uploading PDF materials performed exactly as intended and showed no critical red flags. Testing shows that the code functions and integrates properly with the GUI, developed with Tkinter, such as buttons, input fields, and navigation elements.

The application managed to handle errors smoothly and ran consistently during test cases without significant errors. The integrity of the program proved to be reliable. When a user inputs invalid or incomplete information, the system will display an appropriate warning message. The system saves the information well, keeping student and course record data intact after restarting the application.

4.2 Test Results

Test No.	Featured	Expected Result	Actual Result	Status
Test 1	Add Course	Courses are saved and displayed in the list	Course added successfully and displayed in the course list	**Passed**
Test 2	Edit Course	Updated course info is reflected in the system	Course info is updated correctly	**Passed**
Test 3	Delete Course	The course is removed from the list and the database	Course removed from list successfully	**Passed**
Test 4	Create Account	Future faculty can create an account, as can students.	Form displayed and accepted input	**Passed**
Test 5	Enroll Course	The student is enrolled, and the course appears in the dashboard	Enrolled and showed in list	**Passed**
Test 6	Unenroll Course	The student is removed from the course	Unenrolled and removed from list	**Passed**

Test 7	Forgot Password	Forgot password will work with a code sent to your email.	Worked for registered account	**Passed**
Test 8	Edit Course PDF	The editing course will work with deleting PDF and changing features.	PDF replaced successfully	**Passed**
Test 9	Upload PDF	Uploading a PDF will work as well as multiple uploads of a PDF	PDF uploaded successfully	**Passed**
Test 10	Download PDF	Students can download the PDF	Downloaded successfully	**Passed**
Test 11	Invalid Input	A warning message is displayed	Error message displayed	**Passed**
Test 12	Data Persistence	Data remains after restarting the app	Data retained after restart	**Passed**
Test 13	Log out	User session ends, and the window closes	Logged out successfully	**Passed**

4.3 Analysis of results

After testing, the result shows the majority of the core features of the system work exactly as intended.

5. Conclusion

5.1 Summary of Findings

This project aims to develop and design a desktop-based Learning Management System (LMS) built with Tkinter and Python. The system shows how concepts of object-oriented programming (OOP) can be effectively applied, such as inheritance, classes, and encapsulation, to create a well-designed and functioning system. The system performed reliably under testing, demonstrating consistent data persistence and secure management of course and enrollment information. Tkinter was implemented to create a modernized and seamless graphical user interface (GUI), to provide an intuitive and efficient environment for users to navigate and interact with.

5.2 Recommendation

Based on the experience gained from developing DANAS, the group recommends the following improvements for future versions of the system:

- While the system meets its primary objectives, the design still has areas open for further improvement, but it is designed to scale and expand over time. Future development should be focused on strong security procedures to enhance system integrity, to keep data and information safely stored within the system and establish a robust verification of accounts, keeping the system protected from data breach or unauthorized access.

- To benefit the system, replacing local file storage with a proven and reliable online database such as MySQL, that allows for scalability and support for multiple users at the same time. Adding cloud storage would make access and organization to course materials easier.

- Beyond that, adding features such as quizzes, assignment submissions, grading, and announcements would bring DANAS closer to a fully-fledged learning management system, like Canvas or Moodle.

- Lastly, developing a web or mobile version of DANAS would further enhance its flexibility in terms of accessibility and usability, to allow users to access it more conveniently, anytime or anywhere.